

# MINOR PROJECT – 4

CI/CD Pipeline using GitHub Actions

Docker Image Build and Docker Hub Deployment

|                        |                                     |
|------------------------|-------------------------------------|
| <b>Student</b>         | Dalia Mahata                        |
| <b>Project</b>         | CI/CD Pipeline using GitHub Actions |
| <b>Tools Used</b>      | GitHub Actions, Docker, Docker Hub  |
| <b>Submission Date</b> | 15 September 2026                   |

This report presents the implementation and verification of a simple CI/CD pipeline that pushes source code to GitHub, automatically builds a Docker image using GitHub Actions, and publishes the image to Docker Hub.

# 1. Introduction

Continuous Integration and Continuous Delivery (CI/CD) automate the process of building and delivering software whenever changes are pushed to a source-code repository. In this project, GitHub Actions is used as the automation platform, Docker is used to package the application into a container image, and Docker Hub is used as the image registry.

The project uses a small Python application as the example workload. The source code and Dockerfile are stored in a GitHub repository. A GitHub Actions workflow is triggered when code is pushed to the main branch. The workflow checks out the code, logs in securely to Docker Hub using repository secrets, builds the Docker image, and pushes the image with the latest tag.

## 2. Objectives

- Push the project source code and Dockerfile to GitHub.
- Configure GitHub Actions for automatic CI/CD execution.
- Build the Docker image automatically from the Dockerfile.
- Authenticate securely with Docker Hub using GitHub repository secrets.
- Push the generated Docker image to Docker Hub.
- Pull and run the published image to verify successful deployment.

## 3. Project Configuration

| Component             | Configuration                                      |
|-----------------------|----------------------------------------------------|
| Application           | Python sample application                          |
| GitHub Repository     | daliamahata0505-hash/github-actions-docker-project |
| Docker Hub Repository | daliamahata/github-actions-docker-project          |
| Image Tag             | latest                                             |
| Workflow Trigger      | Push to main branch                                |
| Docker Image          | daliamahata/github-actions-docker-project:latest   |

## 4. GitHub Actions Implementation Evidence

The project source code was first committed to the GitHub repository. The repository contains the Python application, Dockerfile and requirements file. A workflow file named **docker.yml** was then added under **.github/workflows/**.

The workflow is configured to run automatically on every push to the **main** branch. It checks out the repository, authenticates to Docker Hub using the stored GitHub secrets, and uses Docker's build-and-push action to build and publish the image.

### Workflow structure

```
name: Docker CI/CD
Trigger: push to main
Job: build-and-push
Runner: ubuntu-latest
Actions: checkout, Docker login, Docker build and push
```

The required repository secrets are **DOCKERHUB\_USERNAME** and **DOCKERHUB\_TOKEN**. The token is kept private and is referenced by the workflow through GitHub Secrets.

## 4. GitHub Actions Implementation Evidence (continued)

The following evidence shows the GitHub Actions workflow execution. The **build-and-push** job completed successfully with a green check mark.

The screenshot shows the GitHub Actions interface for a workflow named "Add GitHub Actions Docker CI/CD pipeline #1". The workflow is triggered by a push to the main branch. The status is "Success", the total duration is 25s, and there is 1 artifact. The workflow file is named "docker.yml" and is triggered on push. The "build-and-push" job is shown as completed successfully with a green checkmark and a duration of 21s. Below the job status, there is a "build-and-push summary" section titled "Docker Build summary". This section provides a detailed look at the build, including a link to download the build record archive and import it into Docker Desktop's Builds view. The build record is named "daliamahata0505-hash-github-actions-docker-project-B545I9.dockerbuild" (15.57 KB - includes 1 build record). A link "Let us know" is provided for feedback. At the bottom, there is a table with columns: ID, Name, Status, Cached, and Duration.

| ID | Name | Status | Cached | Duration |
|----|------|--------|--------|----------|
|----|------|--------|--------|----------|

Figure 1: GitHub Actions — build-and-push workflow completed successfully.

## 5. Docker Hub Verification

After the GitHub Actions workflow completed, the Docker image was published to the Docker Hub repository. The repository contains the **latest** tag for the image **daliamahata/github-actions-docker-project**.

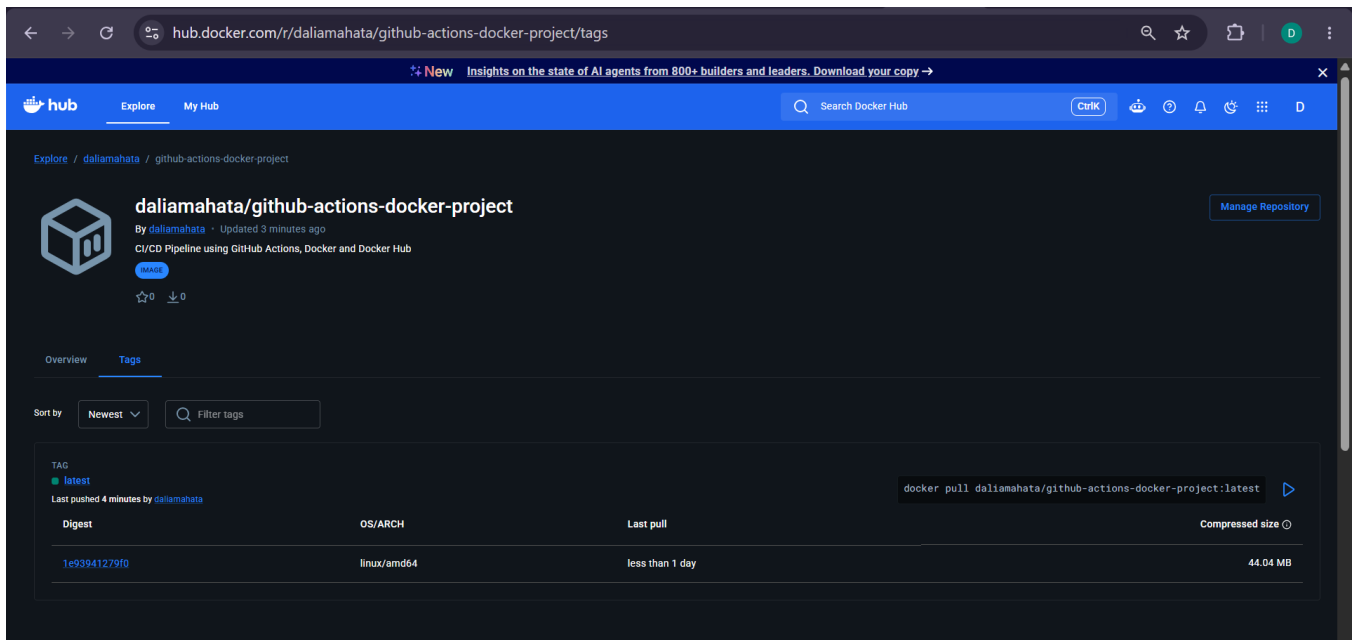


Figure 2: Docker Hub — published image with the latest tag.

## 5. Docker Hub Verification (continued)

The published image was pulled from Docker Hub using Docker CLI. The successful pull confirms that the image pushed by GitHub Actions is available from the registry.

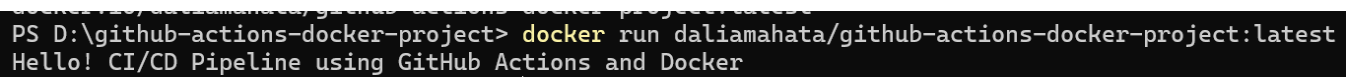
### Command used:

```
docker pull daliamahata/github-actions-docker-project:latest
```

The image was then executed locally using the same Docker Hub image reference.

### Command used:

```
docker run daliamahata/github-actions-docker-project:latest
```

A terminal window screenshot showing a PowerShell prompt. The user enters the command 'docker run daliamahata/github-actions-docker-project:latest'. The output of the command is 'Hello! CI/CD Pipeline using GitHub Actions and Docker'.

```
PS D:\github-actions-docker-project> docker run daliamahata/github-actions-docker-project:latest
Hello! CI/CD Pipeline using GitHub Actions and Docker
```

Figure 3: Docker image pulled and executed successfully from Docker Hub.

## 6. CI/CD Validation and Deployment Results

The CI/CD pipeline was validated from source-code push through container execution. The workflow was triggered after the workflow file was pushed to the main branch. GitHub Actions successfully completed the build-and-push job, and the Docker image became available in Docker Hub.

### Validation steps performed:

1. Git repository initialized and source files committed.
2. Project pushed to GitHub main branch.
3. GitHub Actions workflow added under `.github/workflows/docker.yml`.
4. Docker Hub credentials supplied through GitHub repository secrets.
5. GitHub Actions build-and-push job completed successfully.
6. Docker Hub latest image verified.
7. Image pulled successfully using Docker CLI.
8. Container executed successfully and displayed the expected application message.

### Important commands used

```
git add .github/workflows/docker.yml
git commit -m "Add GitHub Actions Docker CI/CD pipeline"
git push

docker pull daliamahata/github-actions-docker-project:latest
docker run daliamahata/github-actions-docker-project:latest
```

### Final output verified:

**Hello! CI/CD Pipeline using GitHub Actions and Docker**

## 7. Conclusion

The project successfully demonstrates a complete beginner-level CI/CD pipeline using GitHub Actions, Docker and Docker Hub. The application source code was pushed to GitHub, a GitHub Actions workflow automatically built the Docker image, and the resulting image was pushed to Docker Hub.

The published image was independently pulled from Docker Hub and executed successfully. The successful GitHub Actions run, Docker Hub latest image, Docker pull result and Docker run output together provide evidence that the required CI/CD workflow was implemented and verified.

## 8. References

- GitHub Actions documentation
- Docker documentation
- Docker Hub documentation
- GitHub repository: [daliamahata0505-hash/github-actions-docker-project](#)
- Docker Hub repository: [daliamahata/github-actions-docker-project](#)

— End of Report —